



CORPORACIÓN UNIVERSITARIA DEL HUILA – CORHUILA

Comparación de Escenarios de Arquitectura (By Module All Project)

Integrantes:

Bedoya Montealegre Brayan Smith

Facultad De Ingeniería, Corporación Universitaria del Huila-CORHUILA

Arquitectura de software

Docente: Jesús Ariel González Bonilla

20 de septiembre del 2025

Comparación de Escenarios

By Module

organiza el proyecto de forma modular por dominio. Cada módulo, como *Geolocation*, *Infrastructure* o *Security*, contiene todas las capas necesarias para funcionar de manera independiente, incluyendo *Entity*, *IRepository*, *IService*, *Service*, *Controller*, *IDTO* y *DTO*. Además, existe un paquete de utilidades compartido (*Utils*). Este enfoque ofrece una alta cohesión por dominio, ya que todo lo relacionado con un módulo está encapsulado en un solo lugar. Gracias a esto, es más fácil escalar por módulos, distribuir equipos para trabajar en diferentes áreas y mantener el sistema, ya que los cambios en un dominio no afectan directamente a otros. También resulta “microservicios friendly”, porque cada módulo puede evolucionar de forma independiente hasta convertirse en un servicio aislado, y ayuda a reducir acoplamientos cruzados. Sin embargo, entre sus desventajas se encuentran la duplicación de patrones, ya que cada módulo debe implementar sus propias capas; la curva de aprendizaje más alta para nuevos desarrolladores, que deben comprender la estructura de cada módulo; y un mayor overhead en proyectos pequeños, donde esta organización puede sentirse demasiado pesada.

By Layer (All Project)

adopta un enfoque clásico de arquitectura en capas. Aquí se centralizan todas las entidades, repositorios, servicios, controladores y DTOs en carpetas globales, y dentro de cada capa se subdividen los dominios como *Geolocation*, *Infrastructure* o *Security*. Este enfoque es mucho más simple y conocido, lo que lo hace ideal para equipos con menos experiencia y proyectos donde la curva de aprendizaje debe ser baja. Además, evita la duplicación de estructuras porque cada capa existe solo una vez en todo el proyecto, lo que reduce el código repetitivo y facilita la implementación rápida en aplicaciones pequeñas o medianas. No obstante, sus principales desventajas están relacionadas con la cohesión y la escalabilidad. La funcionalidad de un mismo dominio queda dispersa entre distintas capas, lo que dificulta el mantenimiento. Además, existe un mayor riesgo de acoplamiento porque cambios en una capa pueden afectar a todos los dominios. Migrar a microservicios desde esta estructura no es sencillo, ya que requeriría una reestructuración significativa, y el crecimiento del proyecto puede volverse difícil de manejar en equipos grandes.

Proyecto ideal

El Escenario 1 se recomienda para aplicaciones grandes y de largo plazo, especialmente aquellas que requieren escalabilidad y podrían migrar a microservicios en el futuro. También es una buena elección cuando los equipos de trabajo están distribuidos y cada uno se hace cargo de un dominio específico, o en sistemas donde ciertos módulos pueden tener vida independiente, como un servicio de geolocalización. En contraste, el Escenario 2 se ajusta mejor a proyectos pequeños o medianos, con equipos reducidos que necesitan simplicidad y rapidez en el desarrollo, y donde no se espera un crecimiento exponencial en funcionalidades o usuarios. Aquí, la prioridad es entregar valor rápidamente antes que optimizar para escalabilidad futura.

Consumo de recursos de infraestructura

El escenario 1 requiere un mayor consumo inicial, ya que cada módulo puede cargar su propio contexto, servicios y dependencias. Sin embargo, es mucho más eficiente cuando se necesita escalar horizontalmente, porque cada módulo puede desplegarse de forma independiente y consumir solo lo que necesita. Este enfoque es óptimo para arquitecturas en la nube y entornos de contenedores como Docker o Kubernetes. En cambio, el Escenario 2 tiene un menor consumo inicial al ser un único proyecto más monolítico con menos overhead. La desventaja es que resulta más difícil de optimizar, ya que todo el sistema debe desplegarse completo, incluso si solo se requiere una parte, y ofrece menos flexibilidad en entornos distribuidos o de microservicios.

Construcción del esqueleto del proyecto en spring boot

El proyecto del sistema académico está diseñado para gestionar de manera integral todos los aspectos de una institución educativa, desde la administración de estudiantes y cursos hasta la gestión financiera y de seguridad. Su finalidad es ofrecer una plataforma robusta, escalable y modular, que permita la gestión eficiente y organizada de todos los procesos académicos. Dado que el sistema se proyecta para ser amplio y abarcar múltiples dominios y departamentos, la estructura By Module es la más adecuada. Esta organización permite dividir el sistema en módulos independientes, cada uno enfocado en un dominio específico, lo que facilita la colaboración entre equipos, la escalabilidad y el mantenimiento a largo plazo.

Estructura By Module:

En la estructura By Module, organizo el proyecto en módulos independientes, cada uno con sus propias capas: entidades, repositorios, servicios, controladores, DTOs e interfaces DTO. De esta forma, cada dominio (como estudiantes, cursos, administración, finanzas, seguridad) se mantiene aislado, lo que facilita la modularidad y la escalabilidad. Esto es ideal para proyectos grandes, donde distintos equipos pueden trabajar en paralelo sin interferir entre sí.

Estructura All Project (By Layer):

En la estructura All Project, organizo el proyecto por capas globales, es decir, todas las entidades, repositorios, servicios y controladores se encuentran en un solo nivel. Luego, dentro de cada capa, organizo las subcarpetas por dominio. Esto permite una visión más homogénea y es más sencillo para proyectos más pequeños o cuando se prefiere una estructura menos fragmentada. Es ideal cuando la colaboración es más directa y el proyecto no se va a expandir demasiado.

Justificación para escoger By Module

El enfoque By Module resulta más adecuado para un Sistema de Gestión Académica porque este tipo de aplicación es grande, complejo y en constante evolución. A diferencia de proyectos pequeños donde prima la simplicidad, en este escenario la prioridad es la escalabilidad, mantenibilidad y la separación de responsabilidades.

1. **Alta cohesión por dominio**

Cada dominio (estudiante, curso, académico, finanzas, seguridad, etc.) encapsula sus propias entidades, repositorios, servicios y controladores. Esto significa que todos los elementos relacionados con un mismo contexto están juntos, lo que reduce la dispersión del código y facilita su comprensión.

2. **Escalabilidad a largo plazo**

En un sistema académico, es común que con el tiempo se agreguen nuevos módulos (ejemplo: biblioteca, becas, investigación). Con By Module, añadir un módulo nuevo implica crear un paquete independiente sin afectar la estructura existente.

3. **Facilidad de mantenimiento y evolución**

Si surge un cambio en el dominio “student” (ejemplo: nuevas reglas de inscripción), el impacto se concentra dentro de ese módulo, minimizando el riesgo de afectar otras partes del sistema.

4. **Preparación para microservicios**

La estructura modular es un paso natural hacia una arquitectura de microservicios. Si en el futuro se quiere extraer, por ejemplo, el módulo de “payments” o “security” como servicio independiente, el trabajo será mucho más sencillo.

5. **Trabajo en equipos distribuidos**

En una universidad real, los equipos de desarrollo suelen estar divididos por áreas de negocio. Con **By Module**, un equipo puede trabajar en el módulo “academic” y otro en “finance” sin pisarse ni generar demasiados conflictos en el repositorio.

6. **Control de dependencias**

Evita el acoplamiento excesivo. Cada módulo maneja sus propias capas y depende solo de lo necesario, reduciendo la complejidad de interacciones entre dominios.